taking advantage of completely custom model binding. To add custom binding for a parameter type, you must implement one of the following two `static BindAsync` methods in your type `T`:

```
public static ValueTask<T?> BindAsync(HttpContext context);
public static ValueTask<T?> BindAsync(
HttpContext context, ParameterInfo parameter);
```

Both methods accept an `HttpContext`, so you can extract anything you need from the request. But the latter case also provides reflection details about the parameter you're binding. In most cases the simpler signature should be sufficient, but you never know!

Listing 7.8 shows an example of using `BindAsync` to bind a record to the request body by using a custom format. The implementation shown in the listing assumes that the body contains two `double` values, with a line break between them, and if so, it successfully parses the `SizeDetails` object. If there are any problems along the way, it returns `null`.

Listing 7.8 Using `BindAsync` for custom model binding

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapPost("/sizes", (SizeDetails size) => $"Received {size}");        ❶

app.Run();

public record SizeDetails(double height, double width)                  ❷
{                                                                       ❷
    public static async ValueTask<SizeDetails?> BindAsync(              ❷
        HttpContext context)                                            ❷
    {
        using var sr = new StreamReader(context.Request.Body);          ❸

        string? line1 = await sr.ReadLineAsync(context.RequestAborted); ❹
        if (line1 is null) { return null; }                             ❺
```